Zellic

Prepared for
Rachel
Scroll Foundation

Prepared by
Evan Hwang
Xiaochuan Yu
Zellic

March 18, 2026

Private Transfer With USX

Smart Contract Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About Private Transfer With USX 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 9

2.5. Project Timeline 10

3. Detailed Findings **10**

3.1. Both `usxReceiver` and `usdcReceiver` can be controlled by the user 11

3.2. The function `rebalanceBySwapping` does not enforce authorization and `supportedSwapRouters` 13

3.3. Incorrect usage of `mintUSX` in `rebalanceByMinting` 14

3.4. Stolen stale allowance 15

3.5. Potential desynchronization of `encryptedReceiver` and `actualReceiver` when backend is compromised 16

3.6. Insufficient test coverage 18

4. Discussion **19**

4.1. Both native and ERC-20 tokens accepted by `transferToken` 20

4.2. Deployment configuration creates spam risk 20

5. Threat Model **20**

5.1. Contract: PrivateGatewayCloak 21

5.2. Contract: PrivateGatewayScroll 25

5.3. Contract: USXRebalancer 28

6. Assessment Results **30**

6.1. Disclaimer 31

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Scroll Foundation from March 11th to March 13th, 2026. During this engagement, Zellic reviewed Private Transfer With USX's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can an attacker drain the tokens in the gateway?
- Can an attacker freeze the tokens in the gateway?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Private Transfer With USX contracts, we discovered six findings. One critical issue was found. One was of high impact and four were of medium impact.

Scroll Foundation implemented fixes that address all findings with the exception of 3.5 and 3.6. For which they provided more context on why those fixes were not completely addressed at this time.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Scroll Foundation in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	1
Medium	4
Low	0
Informational	0

2. Introduction

2.1. About Private Transfer With USX

Scroll Foundation contributed the following description of Private Transfer With USX:

> The system implements an end-to-end private bridge workflow: encrypted recipient routing, cross-domain confirmation, role-gated release, and liquidity rebalancing.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Private Transfer With USX Contracts

Type	Solidity
Platform	EVM-compatible
Target	usx-contracts
Repository	https://github.com/scroll-tech/usx-contracts ↗
Version	`d69433e75e37cab22defbf80b1fd89c4d6a526cc`
Programs	`src/cloak/**/*.sol`
Target	usx-contracts
Repository	https://github.com/scroll-tech/usx-contracts ↗
Version	`Only changes between 0eb4fa4..bba7dcf`
Programs	`src/**/*.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of three person-days. The assessment was conducted by two consultants over the course of three calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io

The following consultants were engaged to conduct the assessment:

Evan Hwang
Engineer
evan@zellic.io

Xiaochuan Yu
Engineer
xiaochuan@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 11, 2026	Kick-off call
March 11, 2026	Start of primary review period
March 13, 2026	End of primary review period

3. Detailed Findings

3.1. Both `usxReceiver` and `usdcReceiver` can be controlled by the user

Target	PrivateGatewayScroll		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

In `PrivateGatewayScroll::transferToken`, the function accepts both `usxReceiver` and `usdcReceiver` as parameters. The exact same addresses can be used for both parameters.

```
function transferToken(
    address token,
    uint256 swapAmount,
    address swapRouter,
    bytes memory swapData,
    EncryptedReceiver memory usxReceiver,
    EncryptedReceiver memory usdcReceiver
)
```

Since both addresses are user-controlled inputs, users can swap USDC to USX on L2 and then receive the original amount of USDC on L2.

Impact

The caller is able to receive both tokens without actually providing the corresponding USDC amount. This allows the caller to drain the USDC balance of the gateway contract in cloak.

Recommendations

We recommend removing the `usdcReceiver` parameter from the `transferToken` function and setting it to an admin-controlled address.

Remediation

This issue has been acknowledged by Scroll Foundation, and fixes were implemented in the following commits:

- 27a4597e ↗

- c303fda6 ↗

3.2. The function `rebalanceBySwapping` does not enforce authorization and `supportedSwapRouters`

Target	USXRebalancer		
Category	Completeness	**Severity**	High
Likelihood	High	**Impact**	High

Description

As of the time of writing, `rebalanceBySwapping` can be called by any user. This function is intended to be restricted to the owner for rebalancing the USX balance.

And unlike PrivateGatewayScroll, which explicitly validates swap routers,

```
if (!supportedSwapRouters.contains(swapRouter)) {
    revert ErrorSwapRouterNotSupported();
}
```

the `rebalanceBySwapping` flow does not appear to perform a similar check.

Impact

Any user can execute the rebalance flow using arbitrary contracts and calldata. This enables an attacker to profit by swapping the contract's USDC for USX.

Recommendations

We recommend adding a check to ensure that the function is only callable by the owner and the swap router is included in `supportedSwapRouters` before executing the swap.

Remediation

This issue has been acknowledged by Scroll Foundation, and a fix was implemented in commit `27a4597e` ↗.

3.3. Incorrect usage of `mintUSX` in `rebalanceByMinting`

Target	USXRebalancer		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The `rebalanceByMinting` flow calls `mintUSX` after approving USDC:

```
IERC20(USDC).forceApprove(USX, amountUSDC);
IUSX(USX).mintUSX(address(this), amountUSDC);
```

However, `mintUSX` in USX.sol does not accept USDC as input and is restricted to the treasury role.

```
function mintUSX(address _to, uint256 _amount) public onlyTreasury {
    _mint(_to, _amount);
}
```

This function simply mints USX and does not transfer or consume USDC. Additionally, since the function is protected by `onlyTreasury`, and the treasury address is immutable in the deployed USX contract, this call would revert unless the rebalancer contract itself is configured as the treasury.

Impact

The function does not consume the approved USDC, and it mints only raw `amountUSDC` units rather than the expected 18-decimal equivalent. And `rebalanceByMinting` is effectively unusable since the call to mint will revert unless the rebalancer contract is configured as the treasury.

Recommendations

Use `deposit` instead of `mintUSX`.

Remediation

This issue has been acknowledged by Scroll Foundation, and a fix was implemented in commit 27a4597e ↗.

3.4. Stolen stale allowance

Target	PrivateGatewayScroll		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

Users can swap supported tokens to USDC before bridging. Because `swapData` is user-supplied, a stale token amount may remain unused during the swap. The allowance is not cleared afterward.

```
uint256 usdcBefore = IERC20(USDC).balanceOf(address(this));
if (token != address(0)) {
    IERC20(token).forceApprove(spenders[swapRouter], swapAmount);
}
(bool success, ) = swapRouter.call{value: msg.value}(swapData);
if (!success) revert ErrorSwapFailed();
uint256 usdcAfter = IERC20(USDC).balanceOf(address(this));
uint256 usdcAmount = usdcAfter - usdcBefore;
```

As a result, if it consumes only part of the input amount, the unused portion can remain trapped in PrivateGatewayScroll.

Impact

The configured spender may still retain approval to spend some or all of the leftover token balance. An attacker could drain stale allowance by passing a larger amount in the swap data than the original `swapAmount`.

Recommendations

We recommend clearing the allowance after the swap and asserting that all tokens are fully consumed.

Remediation

This issue has been acknowledged by Scroll Foundation, and a fix was implemented in commit `27a4597e` ↗.

3.5. Potential desynchronization of `encryptedReceiver` and `actualReceiver` when backend is compromised

Target	PrivateGatewayScroll		
Category	Completeness	**Severity**	Critical
Likelihood	Low	**Impact**	Medium

Description

In `PrivateGatewayCloak::confirmDeposit`, it calculates the hash with current nonce, encrypted receiver, `keyId` and amount to be transfer. And in `PrivateGatewayCloak::withdrawUSX`, it verifies the hash with the same parameters then withdraw the USX to the `actualReceiver`.

`withdrawUSX` has authorization check to ensure only `WITHDRAW_USX_ROLE` can call it, but currently there is no check against the `actualReceiver`.

```
function withdrawUSX(
    uint256 nonce,
    bytes memory encryptedReceiver,
    uint256 keyId,
    uint256 amountUSDC,
    address actualReceiver
) external onlyRole(WITHDRAW_USX_ROLE) nonReentrant {
    bytes32 hash = keccak256(
        abi.encode(nonce, encryptedReceiver, keyId, amountUSDC)
    );
    if (!confirmedDeposits[hash]) {
        revert ErrorDepositNotConfirmed();
    }
    // [...]
    IERC20(USX).forceApprove(erc20Gateway, amountUSX);
    IL2ERC20GatewayValidium(erc20Gateway).withdrawERC20(
        USX,
        actualReceiver,
        amountUSX,
        0
    );
    // [...]
}
```

Impact

If the backend is compromised, the attacker may abuse the function to withdraw to an address they control.

Recommendations

Currently we recommend restricting the function with multisig role to reduce the risk of potential abuse.

Remediation

This issue has been acknowledged by Scroll Foundation, and they have provided the following response:

> The actualReceiver not being verified on-chain is a deliberate trust tradeoff we made for this version. In production the WITHDRAW_USX_ROLE will sit behind a multisig so even if the back-end gets compromised the damage is limited. On-chain verification is something we want to look into down the line but won't make it into this release.

3.6. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	**Severity**	Medium
Likelihood	N/A	**Impact**	Medium

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test suite consists only of unit tests; there are no integration tests for each bridge deployment (e.g., Scroll and Cloak gateways, messenger flows, and cross-chain deposit/withdrawal sequences). The tests set up a minimal environment and check for intended behavior, but they do not cover all edge cases or scenarios where a transaction should fail or revert.

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend building a rigorous test suite that includes all contracts to ensure that the system operates securely and as intended. Improvements can be made to:

- integration testing: end-to-end flows for each bridge deployment (Scroll and Cloak gateways, messenger, deposit/confirm/withdraw sequences) to ensure cross-chain behavior matches specification.

Unit test cases that are missing include (but are not necessarily limited to) those documented in section 5. ↗.

Remediation

This issue has been acknowledged by Scroll Foundation, and they have provided the following response:

> We're actively expanding our test suite and will continue building out integration tests post-launch. This is an ongoing effort rather than a one-time fix.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Both native and ERC-20 tokens accepted by `transferToken`

Depending on whether the token address is zero, `transferToken` accepts both `msg.value` and ERC-20 transfers. This behavior may introduce ambiguity in usage and could lead to accidental misuse if both paths are possible simultaneously. We recommend to assert `msg.value == 0` when using ERC-20 tokens.

4.2. Deployment configuration creates spam risk

In the deployment configuration as of the time of writing, PrivateGatewayScroll is initialized with `_minUSDCAmount = 0`, which means the gateway accepts zero-amount transfers and emits confirmations for them. A caller may spam with zero-amount transfer confirmations. We recommend setting a nonzero minimum USDC amount to mitigate this risk.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Contract: PrivateGatewayCloak

Function: `confirmDeposit(uint256 nonce, bytes encryptedReceiver, uint256 keyId, uint256 amountUSDC)`

The function `confirmDeposit` records that a deposit was confirmed on the remote side after authenticating both the messenger and the remote counterpart gateway.

Inputs

- `nonce`
  - **Control**: Controlled by the authenticated cross-domain message sender.
  - **Constraints**: Only accepted when `msg.sender == messenger` and `xDomainMessageSender() == counterpart`; uniqueness is enforced only through the full deposit hash.
  - **Impact**: Contributes to the confirmation hash that later unlocks `withdrawUSX()`.
- `encryptedReceiver`
  - **Control**: Controlled by the authenticated cross-domain sender.
  - **Constraints**: No.
  - **Impact**: Becomes part of the deposit identity and is emitted in the confirmation event.
- `keyId`
  - **Control**: Controlled by the authenticated cross-domain sender.
  - **Constraints**: No.
  - **Impact**: Becomes part of the deposit identity used by later withdrawal checks.
- `amountUSDC`
  - **Control**: Controlled by the authenticated cross-domain sender.
  - **Constraints**: No.
  - **Impact**: Determines the confirmed amount and the later USX amount derived in `withdrawUSX()`.

Branches and code coverage

Intended branches

- ☑ A valid messenger call from `counterpart` should store the deposit hash and emit `DepositConfirmed`.

Negative behavior

- ☑ Revert if the direct caller is not the trusted messenger.
- ☑ Revert if the messenger reports an `xDomainMessageSender()` different from `counterpart`.
- ☑ Revert if the same `(nonce, encryptedReceiver, keyId, amountUSDC)` tuple is confirmed twice.

Function: `rebalance()`

The function `rebalance` bridges the contract's entire USDC balance to the configured rebalancer.

Inputs

- None.

Branches and code coverage

Intended branches

- ☑ A caller with `REBALANCE_ROLE` should withdraw the full on-contract USDC balance to `rebalancer` when both are set.
- ☑ The function should approve the gateway and forward the exact current USDC balance with zero fee value.

Negative behavior

- ☑ Revert if the caller lacks `REBALANCE_ROLE`.
- ☑ Revert when the contract holds no USDC.
- ☑ Revert when `rebalancer` has not been configured.

Function: `withdrawTokens(address token, address receiver, uint256 amount)`

The function `withdrawTokens` transfers ETH or arbitrary ERC-20 balances out of the contract, including accidentally held assets and fee inventory.

Inputs

- `token`
  - **Control**: Controlled by an account with `DEFAULT_ADMIN_ROLE`.
  - **Constraints**: `address(0)` selects native ETH; any other address is treated as an ERC-20 token.
  - **Impact**: Determines which asset leaves the contract.
- `receiver`
  - **Control**: Controlled by the admin caller.
  - **Constraints**: No local nonzero or allowlist validation is applied.
  - **Impact**: Receives the withdrawn assets.
- `amount`
  - **Control**: Controlled by the admin caller.
  - **Constraints**: No local balance check is performed before the underlying transfer attempt.
  - **Impact**: Determines how much ETH or ERC-20 is extracted from the contract.

Branches and code coverage

Intended branches

- ☑ An admin should be able to withdraw native ETH when `token == address(0)`.
- ☑ An admin should be able to withdraw arbitrary ERC-20 balances when `token != address(0)`.

Negative behavior

- ☑ Revert if the caller lacks `DEFAULT_ADMIN_ROLE`.
- ☐ Bubble up a revert when the contract lacks sufficient ETH or the ERC-20 transfer fails.
- ☐ Misuse by a trusted admin can drain operational balances because the function intentionally bypasses business-specific checks.

Function: `withdrawUSX(uint256 nonce, bytes encryptedReceiver, uint256 keyId, uint256 amountUSDC, address actualReceiver)`

The function `withdrawUSX` converts a previously confirmed deposit into a USX bridge withdrawal by recomputing the deposit hash, preventing duplicates, and instructing the gateway to withdraw USX to the chosen receiver.

Inputs

- `nonce`

    - **Control**: Controlled by the caller holding `WITHDRAW_USX_ROLE`.
    - **Constraints**: Must match a previously confirmed deposit tuple.
    - **Impact**: Selects which confirmed deposit is being consumed.
- `encryptedReceiver`
    - **Control**: Controlled by the privileged caller.
    - **Constraints**: Must match the bytes used in the prior confirmation hash.
    - **Impact**: Binds the withdrawal to the same deposit identity that was confirmed cross-domain.
- `keyId`
    - **Control**: Controlled by the privileged caller.
    - **Constraints**: Must match the prior confirmation hash.
    - **Impact**: Participates in the consumed deposit identity.
- `amountUSDC`
    - **Control**: Controlled by the privileged caller.
    - **Constraints**: Must match the prior confirmation hash.
    - **Impact**: Determines the bridged USX amount after scaling by 1e12.
- `actualReceiver`
    - **Control**: Controlled by the privileged caller.
    - **Constraints**: No.
    - **Impact**: Receives the withdrawn USX on the gateway path even if it differs from the encrypted-receiver metadata.

Branches and code coverage

Intended branches

- ☑ A caller with `WITHDRAW_USX_ROLE` should consume a confirmed, not-yet-withdrawn deposit and bridge the USX to the supplied receiver.
- ☑ A successful withdrawal should mark the deposit as withdrawn and emit `WithdrawUSX`.

Negative behavior

- ☑ Revert if the caller lacks `WITHDRAW_USX_ROLE`.
- ☑ Revert if the provided tuple was never confirmed.
- ☑ Revert if the provided tuple was already withdrawn.

5.2. Contract: PrivateGatewayScroll

Function: `_transferUSDC(uint256 amount, EncryptedReceiver usxReceiver, EncryptedReceiver usdcReceiver)`

The function `_transferUSDC` validates the active encryption key and minimum amount, deducts fees, deposits USDC through the gateway, increments the nonce, and sends the matching confirmation message to the remote cloak gateway.

Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be at least `minUSDCAmount` before bridging; fee deduction can reduce the net bridged amount.
  - **Impact**: Determines the gross transfer size, fee amount, deposited USDC, and remote confirmation amount.
- `usxReceiver`
  - **Control**: Controlled through internal callers.
  - **Constraints**: No.
  - **Impact**: Provides the encrypted receiver data embedded in the outbound confirmation message.
- `usdcReceiver`
  - **Control**: Controlled through internal callers.
  - **Constraints**: No.
  - **Impact**: Provides the encrypted receiver and key ID used by the gateway deposit.

Branches and code coverage

Intended branches

- ☑ When the latest key and minimum-amount checks pass, the function should deduct fees, deposit the net USDC amount, increment `nonce`, send the confirmation message, and emit `USDCTransferred`.
- ☑ Fees should be capped by `maxFeeAmount` even when the proportional fee would be larger.

Negative behavior

- ☑ Revert if no encryption key exists or if `usxReceiver.keyId` is not the latest key.
- ☑ Revert if `amount` is below `minUSDCAmount` before fee deduction.

Function: `transferToken(address token, uint256 swapAmount, address swapRouter, bytes swapData, EncryptedReceiver usxReceiver, EncryptedReceiver usdcReceiver)`

The function `transferToken` accepts an approved asset or ETH, swaps it into USDC through an approved router, and then forwards the resulting USDC through `_transferUSDC()`.

Inputs

- `token`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be present in `supportedTokens`, unless it is `address(0)` to represent ETH.
  - **Impact**: Determines which asset is pulled or which native-value path is used before the swap.
- `swapAmount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must equal `msg.value` for native-token swaps; otherwise, it must be transferable from the caller.
  - **Impact**: Determines the amount approved to the router spender and the expected swap input size.
- `swapRouter`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be present in `supportedSwapRouters`.
  - **Impact**: Receives the low-level swap call that consumes approved tokens.
- `swapData`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Dictates arbitrary calldata executed on the approved router.
- `usxReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Supplies the encrypted receiver metadata sent to the remote confirmation.
- `usdcReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Supplies the encrypted receiver and key ID used by the gateway deposit.

Branches and code coverage

Intended branches

- ☑ A caller using a supported ERC-20 should transfer in `swapAmount`, execute the approved router call, and bridge the resulting USDC when all downstream checks pass.
- ☑ A caller using native ETH should supply `msg.value == swapAmount`, execute the approved router call, and bridge the resulting USDC.

Negative behavior

- ☑ Revert if `token` is not supported.
- ☑ Revert if `swapRouter` is not supported.
- ☑ Revert if native ETH `msg.value` does not match `swapAmount`.
- ☑ Revert if the router call reverts.

Function: `transferUSDC(uint256 amount, EncryptedReceiver usxReceiver, EncryptedReceiver usdcReceiver)`

The function `transferUSDC` pulls USDC from the caller and routes it through `_transferUSDC()` to bridge the net amount plus the matching confirmation message.

Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be approved for transfer and later satisfy `_transferUSDC()` minimum checks after fees.
  - **Impact**: Defines how much USDC is pulled from the caller before fee assessment.
- `usxReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Its `keyId` must equal the latest registered encryption key; the receiver bytes are not locally validated.
  - **Impact**: Provides the encrypted receiver data that is sent to the remote confirmation path.
- `usdcReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Its `keyId` is not locally checked here; the downstream gateway performs its own validation.
  - **Impact**: Provides the encrypted USDC deposit receiver passed to the gateway.

Branches and code coverage

Intended branches

- ☑ A funded caller should transfer USDC into the contract, bridge the post-fee amount, increment `nonce`, and emit `USDCTransferred` when the latest key and minimum-amount checks pass.
- ☑ The fee charged on a successful transfer should be capped by `maxFeeAmount` when the proportional fee would be larger.

Negative behavior

- ☐ Revert if `safeTransferFrom()` cannot pull the requested USDC from the caller.
- ☑ Revert if no encryption key exists, if `usxReceiver.keyId` is stale, or if the amount is below `minUSDCAmount` inside `_transferUSDC()`.
- ☐ Bubble up a revert if the gateway deposit or messenger call fails.

5.3. Contract: USXRebalancer

Function: `rebalanceByMinting(uint256 amountUSDC, EncryptedReceiver usxReceiver)`

The function `rebalanceByMinting` converts USDC into minted USX, measures the amount received, and bridges that USX to the encrypted receiver.

Inputs

- `amountUSDC`
  - **Control**: Controlled by a caller with `REBALANCE_ROLE`.
  - **Constraints**: The contract must already hold at least this much USDC.
  - **Impact**: Determines how much USDC is approved to the USX contract and how much USX is expected to be minted.
- `usxReceiver`
  - **Control**: Controlled by the privileged caller.
  - **Constraints**: No.
  - **Impact**: Receives the bridged USX through `_transferUSX()`.

Branches and code coverage

Intended branches

- ☑ A caller with `REBALANCE_ROLE` should mint USX against existing USDC inventory and bridge the resulting USX to the supplied encrypted receiver.

- ☑ A successful minting rebalance should deposit the minted amount and leave no residual USX in the contract.

Negative behavior

- ☑ Revert if the caller lacks `REBALANCE_ROLE`.
- ☑ Revert if the contract does not hold enough USDC.

Function: `rebalanceBySwapping(uint256 amountUSDC, address swapRouter, bytes swapData, EncryptedReceiver usxReceiver)`

The function `rebalanceBySwapping` approves USDC to a router, executes arbitrary router calldata, and bridges the acquired USX out.

Inputs

- `amountUSDC`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The contract must already hold at least this much USDC.
  - **Impact**: Determines how much USDC is approved to the router.
- `swapRouter`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Receives the low-level swap call that consumes approved USDC.
- `swapData`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Defines arbitrary calldata executed on the router.
- `usxReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No.
  - **Impact**: Receives the bridged USX through `_transferUSX()`.

Branches and code coverage

Intended branches

- ☑ A caller should approve USDC to the router, execute the router call, and bridge the resulting USX to the supplied encrypted receiver.
- ☑ A successful swap rebalance should deposit the resulting USX and leave no residual

USX in the contract.

Negative behavior

- ☑ Revert if the contract lacks sufficient USDC inventory.
- ☑ Revert if the low-level router call fails.

6. Assessment Results

During our assessment on the scoped Private Transfer With USX contracts, we discovered six findings. One critical issue was found. One was of high impact and four were of medium impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.